SafeRide Architecture Documentation

Frontend Session Management

1. Multi-Tab Logout Synchronization

SafeRide uses a stateless, browser-native event-driven approach to synchronize session state across multiple tabs without incurring API overhead or maintaining persistent WebSocket connections.

Mechanism: `storage` Event Listener

1. When a user explicitly logs out in Tab A, the frontend clears the `saferide_session` object from `localStorage` and triggers the backend `POST /api/v1/auth/logout`.
2. The browser automatically fires a `StorageEvent` to all other open tabs on the same origin (`localhost:5173`).
3. The `AuthContext` provider in Tabs B, C, D, etc., listens to this event. If it detects that `saferide_session` was removed (`newValue === null`), it instantly nullifies the local React context state (`isAuthenticated = false`) and redirects the user to the `/login` screen.
4. **Benefit**: This guarantees 100% immediate logout synchronization across all browser windows with $0 infrastructure cost and zero network polling.

2. OIDC Flow "Flash Cookie" Pattern

To support OAuth 2.0 / OpenID Connect flows with Ory Hydra without exposing ugly or easily tampered `login_challenge` strings in the browser URL address bar, SafeRide utilizes a "Flash Cookie Consumption" pattern.

Mechanism: Read-and-Destroy

1. The backend sets the `login_challenge` in a temporary, non-HttpOnly cookie and redirects the browser to a clean URL: `http://localhost:5173/login`.
2. When the React `LoginPage` mounts, it reads the cookie into its local memory (React State).
3. In the exact same render cycle, it executes: `document.cookie = "login_challenge=; path=/; expires=Thu, 01 Jan 1970 00:00:00 UTC;"`
4. **Benefit**: This cleanly erases the challenge from the browser storage so that it cannot be accidentally reused (which would cause a "Session Expired" error) while allowing the active login page to complete its flow.

3. Frontend JSON Redirect Handling

Since the frontend operates as an SPA communicating with the backend via `fetch` over API endpoints, the final step of the OIDC login acceptance (where the backend must route the user through Hydra Consent) is returned as a JSON payload: `{ "redirectTo": "/api/v1/auth/oauth/login" }`.

The `AuthContext` intercepts this JSON payload and assigns `window.location.href = res.data.redirectTo`. This correctly unloads the React app, allowing the browser to follow the standard OAuth2 302 redirects through Hydra and back to the backend `/callback` to finalize the `HttpOnly` token provisioning.